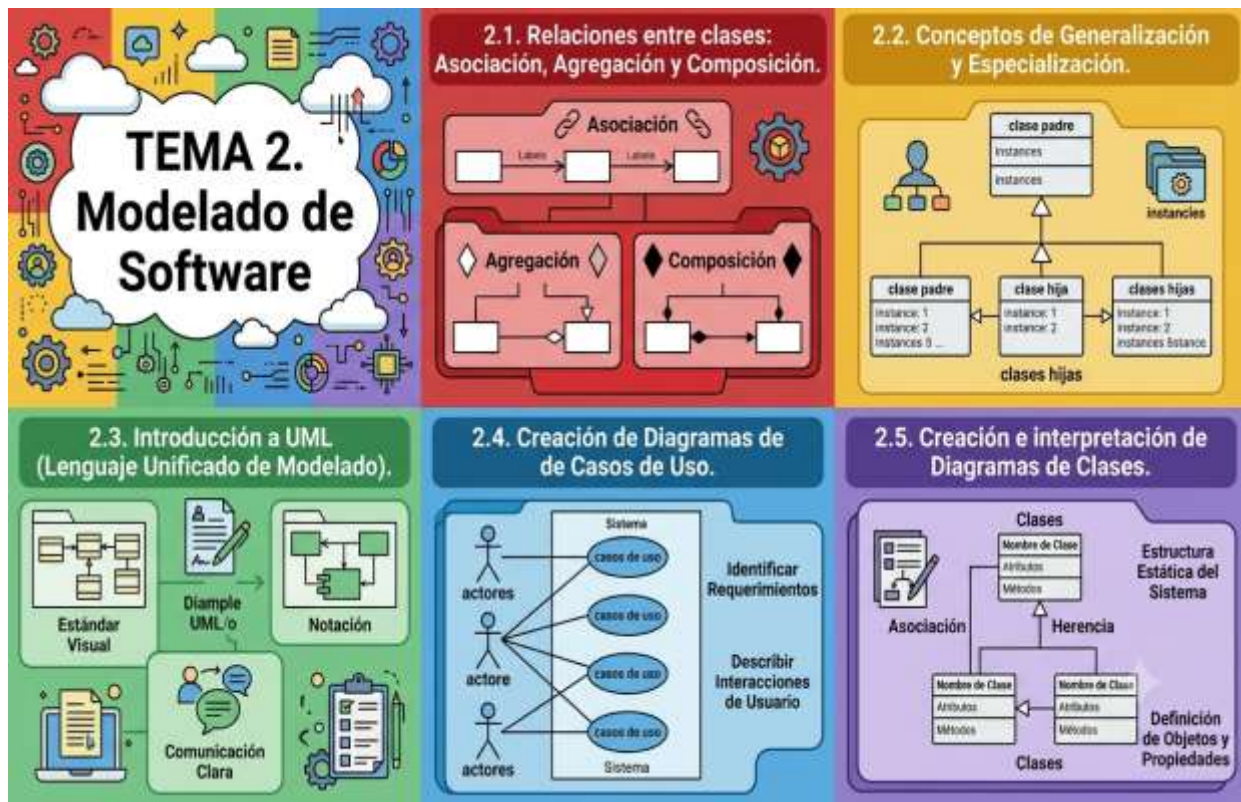


# TEMA N° 2



## TEMA 2 MODELADO DE SOFTWARE



### EN ESTE TEMA SERÁS CAPAZ DE...

1. **Comprender y estructurar visualmente** el diseño de sistemas informáticos antes de escribir una sola línea de código, garantizando que el software resuelva necesidades reales.
2. **Diseñar e interpretar** Diagramas de Clases y Casos de Uso utilizando el Lenguaje Unificado de Modelado (UML), aplicándolos a los escenarios cotidianos de la población de Pailón.
3. **Preparar tu mentalidad** como futuro Técnico en Sistemas Informáticos para planificar proyectos escalables, eficientes y listos para integrarse con bases de datos y arquitecturas web modernas.



## PRÁCTICA



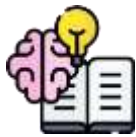
¿Te imaginas construir el nuevo coliseo del Barrio 27 de Mayo sin tener primero los planos arquitectónicos? Seguramente las paredes quedarían chuecas, el techo no encajaría y los materiales se desperdiciarían. En el software ocurre exactamente lo mismo.

Imagina la siguiente situación real:

El CEA Herman Ortiz Camargo acaba de inaugurar su flamante y bien equipado módulo educativo en el Barrio Saó. La dirección ha decidido modernizar la institución y te han encargado, como estudiante de Sistemas Informáticos, planificar una "Plataforma Educativa Institucional". Este sistema debe manejar las inscripciones de los estudiantes que llegan de diferentes zonas: jóvenes trabajadores del comercio formal del Barrio Central Fátima, deportistas del Barrio 24 de Septiembre, padres de familia del Barrio Residencial Norte e incluso vecinos de Las Misiones, San Antonio y San José. Además, el sistema debe incluir un módulo de votación digital para las elecciones institucionales.

Si te sientas ahora mismo frente a la computadora y abres tu editor de código para empezar a programar sin un plan, te perderás en miles de líneas. ¿Cómo se relaciona un estudiante del Barrio María Belén con la materia de Programación? ¿Qué acciones puede realizar el director frente a las que puede realizar un docente? ¿Cómo dibujamos esa estructura en un papel para que todo tu equipo de trabajo lo entienda antes de programar?

Esa necesidad de "dibujar los planos" de nuestro sistema para el CEA antes de construirlo es lo que resolveremos en este tema.



## TEORÍA



En este espacio, como Técnico en Sistemas Informáticos, aprenderás las bases estructurales para diseñar cualquier programa, plataforma o aplicación.

### 2.1. RELACIONES ENTRE CLASES: ASOCIACIÓN, AGREGACIÓN Y COMPOSICIÓN

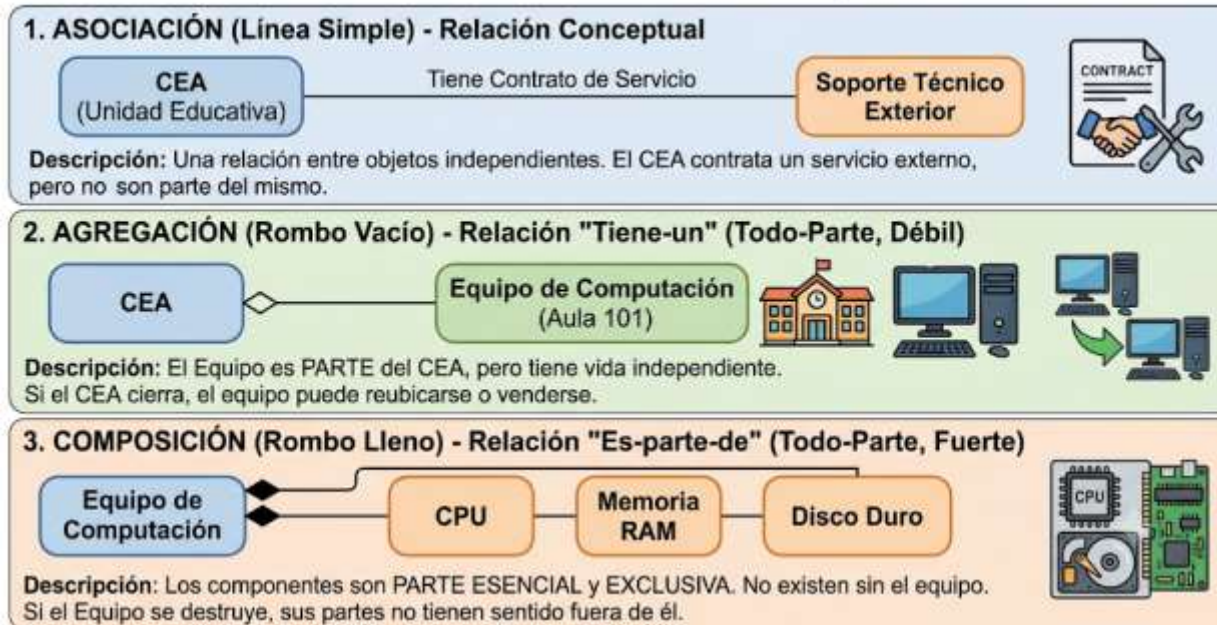
En el mundo real, los objetos no existen de forma aislada; interactúan entre sí. En la programación orientada a objetos (POO), estas interacciones se denominan "Relaciones". Comprenderlas es vital para estructurar tu software.

1. **Asociación:** Es la relación más básica. Dos clases se conectan porque necesitan comunicarse, pero ambas son independientes y pueden existir sin la otra.
  1. *Ejemplo en Pailón:* Un Estudiante y un Docente del CEA. El docente enseña al estudiante, pero si el estudiante deja de asistir, el docente sigue existiendo. Si el docente renuncia, el estudiante sigue siendo estudiante.
2. **Agregación:** Es un tipo de asociación más fuerte, conocida como relación "tiene-un" (has-a), pero las partes pueden seguir existiendo si el contenedor se destruye. Es una relación de "todo-parte" débil.
  1. *Ejemplo en Pailón:* El ModuloEducativo del Barrio Saó y los EquiposDeComputacion. El módulo "tiene" equipos. Si, hipotéticamente, el edificio del módulo fuera demolido, las computadoras pueden ser trasladadas y seguir funcionando en otro lugar (como en la parroquia del Barrio Fátima).
3. **Composición:** Es la relación más fuerte. Es un "todo-parte" estricto. Si la clase principal (el contenedor) deja de existir, las clases contenidas también se destruyen. Su ciclo de vida está atado.
  1. *Ejemplo en Pailón:* Una EleccionInstitucional y un Voto. Si la elección del CEA se anula o se elimina del sistema, todos los votos registrados para esa elección específica dejan de tener



sentido y se destruyen con ella. Un voto no puede existir flotando en el sistema sin pertenecer a una elección.

### DIFERENCIAS VISUALES EN RELACIONES UML: CEA Y EQUIPOS DE COMPUTACIÓN



## 2.2. CONCEPTOS DE GENERALIZACIÓN Y ESPECIALIZACIÓN

Este concepto está íntimamente ligado a lo que en programación conocemos como **Herencia**. Nos permite reutilizar código y organizar nuestras clases desde lo más general hasta lo más específico.

- Generalización (De abajo hacia arriba):** Es el proceso de extraer características compartidas de varias clases específicas para crear una clase general.
- Ejemplo:** Tienes a los vecinos del Barrio Oto Waver que compran en el mercado, a los profesores del CEA, y a los estudiantes de Jardines de Pailón. Todos ellos tienen nombre, carnet de identidad, fecha de nacimiento y dirección. La generalización nos dice: "No repitas esos atributos en cada clase. Crea una clase general llamada Persona que contenga todo eso".
- Especialización (De arriba hacia abajo):** Es el proceso inverso. Tomas una clase general y creas clases específicas que heredan de ella, añadiendo detalles únicos.



1. *Ejemplo:* Partimos de la clase Persona. Especializamos esta clase creando la clase Estudiante (que añade el atributo matricula y el método inscribirModulo()) y la clase Docente (que añade el atributo especialidad y el método subirNotas()).

### 2.3. INTRODUCCIÓN A UML (LENGUAJE UNIFICADO DE MODELADO)

Como Técnico, necesitas comunicarte con otros desarrolladores, clientes o directores de la institución. Si les muestras código puro, no te entenderán. Aquí entra **UML**.

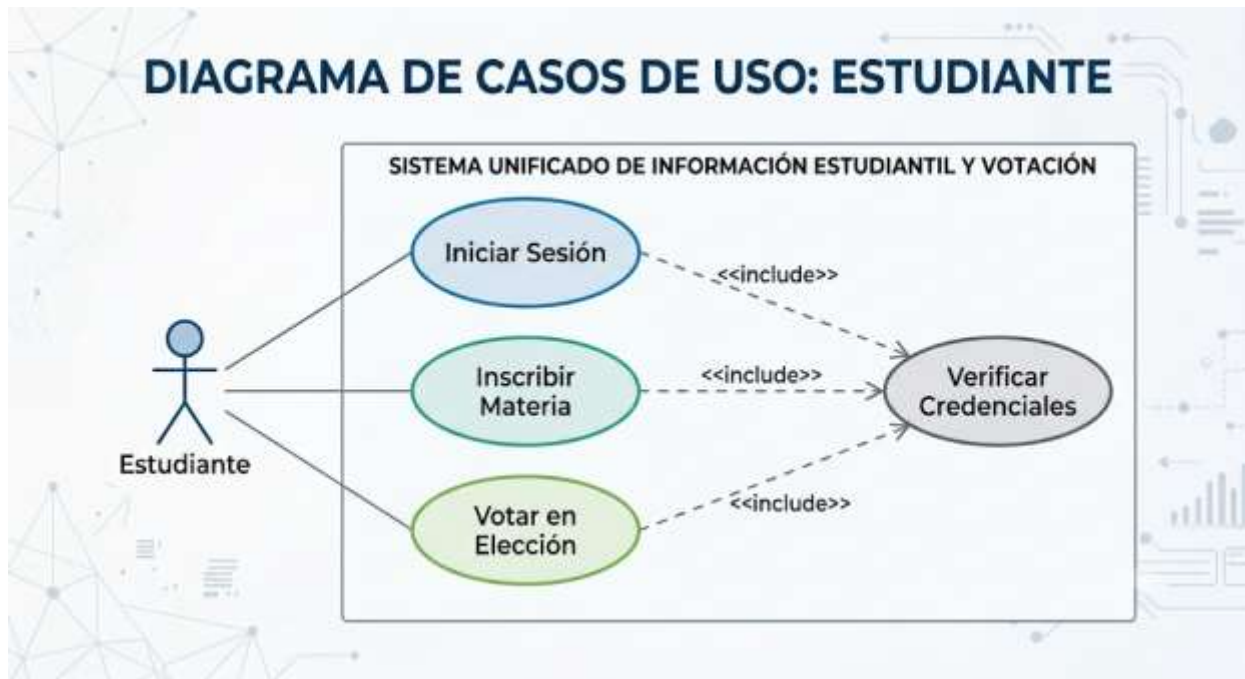
UML no es un lenguaje de programación (no sirve para crear un software funcional por sí solo). Es un lenguaje gráfico y visual para especificar, construir y documentar los artefactos de un sistema de software. Es el estándar mundial. Imagínalo como el idioma universal de los planos arquitectónicos, pero para programas. Con UML, puedes modelar desde un pequeño sistema de ventas para una tienda en el Barrio San Expedito, hasta la base de datos de las elecciones estudiantiles del CEA.

### 2.4. CREACIÓN DE DIAGRAMAS DE CASOS DE USO

Los diagramas de casos de uso son excelentes para responder a la pregunta: "*¿Qué hace el sistema y quién lo usa?*". No muestran *cómo* funciona por dentro, sino *qué* valor aporta desde la perspectiva del usuario.

1. **Actor (Muñeco de palo):** Representa a un usuario o sistema externo que interactúa con nuestra aplicación. Ej: Estudiante, Administrador, Sistema de Impuestos.
2. **Caso de Uso (Óvalo):** Es una acción específica que el actor puede realizar en el sistema. Siempre se escribe con un verbo en infinitivo. Ej: Emitir Voto, Registrar Asistencia, Inscribir Semestre.
3. **Relaciones:** Conectan a los actores con los casos de uso. También existen relaciones entre casos de uso como <<include>> (obligatorio, por ejemplo, "Emitir Voto" *incluye* "Verificar Identidad") y <<extend>> (opcional, "Inscribir Semestre" *se extiende* a "Solicitar Beca" si el estudiante lo requiere).





## 2.5. CREACIÓN E INTERPRETACIÓN DE DIAGRAMAS DE CLASES

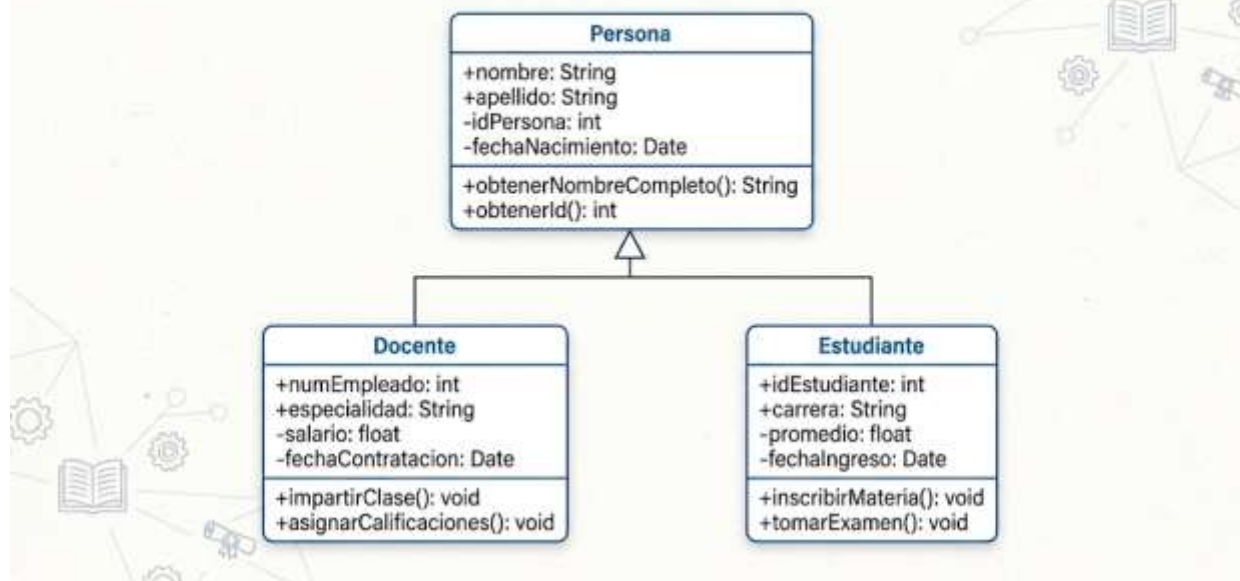
Si los Casos de Uso son la vista externa, el Diagrama de Clases es la vista interna, el esqueleto real de tu código orientado a objetos. Cada rectángulo representa una clase dividida en tres compartimentos:

1. **Nombre de la Clase:** Arriba, centrado y en negrita (ej. Materia).
2. **Atributos (Propiedades):** Las características. Tienen un símbolo de visibilidad: + (público), - (privado), # (protegido). Ej: - nombre: String.
3. **Métodos (Comportamientos):** Las funciones. Ej: + asignarDocente(Docente d): void.

Un buen diagrama de clases te permite pasar directamente al código (ya sea en Python, Java, JavaScript, etc.) porque te indica exactamente qué variables y funciones debes programar, y cómo se enlazan mediante asociación, agregación, composición o herencia.



## DIAGRAMA DE CLASES: JERARQUÍA DE PERSONA



## 2.6. MODELADO ÁGIL Y ARQUITECTURAS EN LA NUBE

En el mercado actual, un Técnico en Sistemas Informáticos no solo diseña sistemas monolíticos antiguos. Hoy, el software se despliega en la nube (plataformas como Render, Railway o Netlify). Al modelar, debemos pensar en "Microservicios". En lugar de un diagrama gigante para todo el CEA, modelamos pequeños sistemas independientes: un sistema de inscripciones por un lado, y un sistema de elecciones institucional por otro. Estos se comunican mediante APIs. Comprender UML te ayuda a trazar las fronteras de estos servicios para no saturar tu servidor y manejar altos volúmenes de datos en la nube.

## 2.7. HERRAMIENTAS MODERNAS Y ASISTENCIA DE INTELIGENCIA ARTIFICIAL EN UML

El dibujo manual de diagramas es excelente para aprender, pero en el entorno profesional, la velocidad es clave. Hoy en día, utilizamos lenguajes de marcado como **PlantUML** o **Mermaid.js**, que nos permiten "programar" el diagrama mediante texto, y una herramienta genera el gráfico automáticamente. Aún mejor, como técnicos modernos, podemos alimentar un *prompt* en una Inteligencia Artificial (como Gemini o ChatGPT) describiendo nuestro sistema en el Barrio Saó, y la IA puede generarnos el código de PlantUML en segundos. Sin embargo, la IA comete errores lógicos; por eso tú, como profesional, debes saber leer UML para corregir e interpretar el resultado que la máquina te da.



## 2.8. MODELADO DE BASES DE DATOS A PARTIR DE UML

El Diagrama de Clases es el paso previo obligatorio para diseñar una Base de Datos Relacional (como PostgreSQL o MySQL). Las clases se convierten en **Tablas**, los atributos privados se convierten en **Columnas**, y las relaciones (asociación, agregación) se transforman en **Llaves Foráneas (Foreign Keys)**. Un excelente modelado UML garantiza que tu base de datos esté normalizada, no tenga redundancia de información y responda rápido cuando los estudiantes consulten sus notas en la plataforma del CEA.

## CIERRE TEÓRICO OBLIGATORIO

### □ *Mitos vs. Realidades en el Modelado de Software*

| Mito                                                                                                                           | Realidad                                                                                                                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mito 1:</b> "Hacer diagramas UML es una pérdida de tiempo, es mejor empezar a programar directamente para terminar rápido." | <b>Realidad:</b> Programar sin modelar causa errores estructurales graves, código espagueti y refactorizaciones costosas. Un buen diseño previo ahorra semanas de programación y depuración, asegurando un despliegue exitoso.                                      |
| <b>Mito 2:</b> "Los diagramas UML deben ser perfectos y seguir reglas rígidas e inamovibles, de lo contrario no sirven."       | <b>Realidad:</b> UML es una herramienta de comunicación ágil. Es preferible un diagrama dibujado a mano en una pizarra o en un cuaderno que tu equipo entienda rápidamente, a un diagrama digital perfecto pero incomprensible. La utilidad prima sobre la rigidez. |

### □ *Glosario de Términos Técnicos*

1. **Actor:** Entidad externa (persona o sistema) que interactúa con nuestra aplicación dentro de un diagrama de casos de uso.
2. **Atributo:** Variable que define las características o propiedades de una clase (ej. el color, el nombre, la edad).
3. **Herencia:** Mecanismo donde una clase hija adquiere las propiedades y métodos de una clase padre, fomentando la reutilización de código.





4. **Método:** Función interna de una clase que define el comportamiento o las acciones que los objetos de dicha clase pueden realizar.
5. **Microservicio:** Arquitectura moderna de software donde una aplicación grande se divide en servicios pequeños e independientes que se comunican entre sí.

#### □ *Ponte a Prueba*

1. Si el edificio de un colegio se destruye, los estudiantes siguen existiendo. En un diagrama UML, ¿qué tipo de relación es esta entre el Edificio y el Estudiante?
  - a) Composición
  - b) Generalización
  - c) Agregación
2. En un Diagrama de Clases, el signo menos (-) delante de un atributo significa que la visibilidad es:
  - a) Pública
  - b) Privada
  - c) Protegida
3. El diagrama que muestra *qué* hace el sistema y *quién* interactúa con él, sin mostrar el código interno, se llama:
  - a) Diagrama de Clases
  - b) Diagrama de Casos de Uso
  - c) Diagrama de Base de Datos



### VALORACIÓN



Todo desarrollo tecnológico impacta en la comunidad. Al modelar un sistema informático, no solo estás organizando código; estás tomando decisiones éticas y ambientales.

Pensemos en Pailón. Si diseñas una plataforma para el CEA Herman Ortiz Camargo sin un buen modelo (Diagrama de Clases deficiente), el software estará sobrecargado y será lento. Esto exigirá procesadores más potentes y servidores en la nube más grandes. A mayor uso de servidores,



mayor consumo de energía eléctrica y mayor huella de carbono, además de que los estudiantes con celulares antiguos (e-waste) no podrán acceder al sistema fluido.

Por otro lado, la **privacidad** es crucial. Si no modelas correctamente la encapsulación (visibilidad privada de atributos) o la base de datos de la plataforma electoral del instituto, los datos personales de los estudiantes de zonas como Ovidio Barberi o San Expedito podrían estar expuestos a ciberataques. Un Técnico en Sistemas Informáticos responsable no solo programa para que "funcione", programa para que el software sea ligero, ético, seguro y respete la privacidad de su comunidad.



## PRODUCCIÓN



### Reto Final de Laboratorio: Diagramando la Plataforma Institucional del CEA.

Tu misión es aplicar lo aprendido usando **PlantUML**, una herramienta basada en texto que genera diagramas de clases, muy utilizada en la industria. Vamos a modelar un pequeño fragmento del sistema de elecciones del CEA.

**Paso 1:** Abre tu navegador y dirígete a un servidor gratuito de PlantUML, como PlantText ([www.planttext.com](http://www.planttext.com)).

**Paso 2:** Borra el código de ejemplo que aparece en la pantalla a la izquierda.

**Paso 3:** Escribe el siguiente código. Lee los comentarios atentamente para entender cómo la teoría se convierte en diagramas.

#### Fragmento de código

```
' Iniciamos el diagrama@startuml
' DEFINICIÓN DE CLASES
' Definimos la clase general usando la palabra clave classclass Persona {
' El signo - indica que es privado (encapsulamiento) - carnetIdentidad: String
- nombreCompleto: String - barrioDeResidencia: String}
' Definimos la clase hija (Especialización)
class Estudiante {
- codigoMatricula: String ' El signo + indica un método público +
emitirVoto(eleccion: Eleccion): void}
class EleccionInstitucional {
```



```
- periodo: String - activa: Boolean + cerrarEleccion(): void + contarVotos(): Integer}
class Voto {
- fechaHora: Date - candidatoElegido: String}
' RELACIONES (La magia de UML en PlantUML)
' 1. HERENCIA: Estudiante hereda de Persona. ' Se usa la flecha con triángulo vacío <|
--Persona <|
-- Estudiante
' 2. ASOCIACIÓN: Un Estudiante participa en una EleccionInstitucional.
' Se usa una línea simple --Estudiante "1" -- "*" EleccionInstitucional : Participa en >
' 3. COMPOSICIÓN: Una ElecciónInstitucional ESTÁ COMPUESTA por Votos.
' Si la elección se anula, los votos desaparecen. ' Se usa el rombo lleno *
--EleccionInstitucional "1" *
-- "*" Voto : Contiene >@enduml
```

**Paso 4:** Haz clic en el botón "Refresh" o "Render" en la herramienta web. Verás cómo, mágicamente, tus líneas de código se transforman en un Diagrama de Clases perfecto y profesional.

**Paso 5:** Analiza la imagen resultante. Verifica dónde está el rombo lleno de composición, cómo se ve la flecha de herencia, y reflexiona: ¿Cómo programarías esto en tu lenguaje backend favorito, como Python o Java? Exporta la imagen resultante para adjuntarla a tu portafolio.

## 5. RECURSOS ADICIONALES

Para profundizar tus habilidades y ser un Técnico altamente competitivo, explora estas herramientas de clase mundial:

1. **PlantUML (Documentación Oficial):** Aprende la sintaxis completa para dibujar diagramas usando solo texto. Esencial para integrarlo con GitHub y documentación de proyectos.
  1. *Enlace recomendado:* <https://plantuml.com/es/class-diagram>
2. **Lucidchart / Draw.io (Diagramación Visual):** Plataformas web gratuitas con interfaz gráfica (arrastrar y soltar) ideales para bosquejos rápidos y colaboración en tiempo real con tu equipo de desarrollo.
  1. *Enlace recomendado:* <https://app.diagrams.net/> (Draw.io)



3. **UML.org (Guías y Estándares):** La página oficial del Object Management Group (OMG), los creadores del estándar UML. Aquí encontrarás las reglas puras del lenguaje si necesitas resolver dudas técnicas avanzadas.

1. *Enlace recomendado:* <https://www.uml.org/>